LANGUAGE REFERENCE · VERSION 0.2.0

Plain

Twenty-nine words, one way to write each thing, and errors that tell you *what to do next* instead of what went wrong internally.

This document is complete. Every word the language has, and every action built into it, is described here — there is no second half kept somewhere else.

29 words · 20 built-in actions · nothing to install
Generated 19 August 2026

Contents

Version 0.2 · Complete. Every word the language has is in this document.

Plain has 29 words and 20 built-in actions. That is the whole language. There is nothing to install, nothing to import, and no second half kept somewhere else.

---

1. The shape of a program

A program is a list of instructions, one per line, run from top to bottom.

```
set name to "Mark"
show "Hello, " + name
```

Four rules cover all of it:

- **One instruction per line.** No semicolons.
- **Indentation is yours.** Blocks are closed by the word `end`, so spacing can never change what a program does. Indent for readability or don't.
- `#` **starts a note.** Everything after it on that line is ignored.
- **Blank lines mean nothing.** Use them freely.

```
# This is a note. The line below is not.
set total to 0
```

---

2. The six kinds of value

KIND	WRITTEN AS	NOTES
Number	`12`, `3.5`, `-8`	One kind only. No separate whole/decimal types.
Text	`"hello"`	Always double quotes.
True/false	`true`, `false`	The only things an `if` will accept.
List	`[1, 2, 3]`	**Items count from 1.**
Record	`{name: "Ada"}`	Named fields.
Nothing	`nothing`	No value yet.

There is a seventh kind you create yourself — an **action**, made with `make`. See section 9.

Inside text you can use `\n` for a new line, `\"` for a quote mark, and `\\` for a backslash.

---

3. `set` and `change` — names and values

`set` creates a name. `change` gives an existing one a new value. They are separate on purpose, so a typo can never quietly create a second variable.

```
set total to 0          # create
change total to 5       # alter
```

Each fails clearly if used the wrong way round:

```
set total to 0
set total to 5
```

Line 2: `total` already exists. To give it a new value, use `change total to 5`.

```
change score to 10
```

Line 1: There's nothing called `score` to change. Create it first with `set score to 10`.

```
set total to 0
change totl to total + 5
```

Line 2: There's nothing called `totl` to change. There is something called `total` — did you mean that?

That last one is the point of the whole design. In most languages it silently makes a second variable and the bug surfaces much later.

There is no `=` in Plain, so `=` and `==` can never be confused.

One name means one thing. `set` fails if the name is already in use anywhere you can see it, including the built-in actions:

```
set count to 3
```

Line 1: `count` is already the name of a built-in action.

Names start with a letter or `_` and may contain letters, digits and `_`. They are case-sensitive.

`change` also reaches inside lists and records:

```
set xs to [10, 20, 30]
change xs[2] to 99         # xs is now [10, 99, 30]

set p to {name: "Ada"}
change p.name to "Grace"
change p.city to "London"  # adds a new field
```

`set` only ever creates a whole new name, so it never has a `.` or `[` after it.

4. `show` — displaying something

```
show "Hello"
show 42
show total
```

Several values separated by commas appear on one line, with a space between:

```
set prices to [10, 20]
show "How many:", count(prices), "Total:", sum(prices)
```

```
How many: 2 Total: 30
```

`show` displays text without its quote marks. Inside a list or record, text keeps its quotes so you can see what is text and what is a number.

5. How things combine — the important part

This is the rule that makes everything else fit together:

> **Anywhere a value is allowed, any expression that produces a value is allowed.**

An **expression** is anything that produces a value: a number, some text, a name, a list, arithmetic, a comparison, a field, a position, or an action call. A **statement** is a whole instruction: `set`, `change`, `show`, `if`, `for each`, `repeat`, `while`, `make`, `give`, `stop`.

So `count(prices)` produces a number, and can go anywhere a number can go:

```
show count(prices)                      # in a show
set how_many to count(prices)           # in a set
if count(prices) is more than 3         # in a test
show "Items: " + count(prices)          # joined onto text
set doubled to count(prices) * 2        # in arithmetic
```

Actions can go inside other actions. Read them from the inside out:

```
show first(sort(prices))
# sort(prices) gives an ordered list
# first(...) then takes its first item
```

```
show round(sum(prices) / count(prices))
# sum gives a total, count gives how many,
# / divides them, round makes it whole
```

Nest as deeply as you like. There is no limit and no special syntax for it.

What does not combine: a statement can never go inside an expression. These are all wrong:

```
set x to show 5           # show is a statement, not a value
set x to (set y to 2)     # set is a statement, not a value
show if true              # if is a statement, not a value
```

Where each thing must produce the right kind of value:

POSITION	MUST BE
`if` and `while` tests	true or false — nothing else is accepted
`for each … in`	a list, or text
`repeat … times`	a number
`[position]`	a number, from 1 upwards

6. Arithmetic and joining

```
show 10 + 5      # 15
show 10 - 5      # 5
show 10 * 5      # 50
show 10 / 4      # 2.5
show 10 % 3      # 1   — the remainder after dividing
```

`+` does three jobs, depending on what is on either side:

```
show 2 + 3                # 5          number plus number
show "a" + "b"            # ab         text joined to text
show "Total: " + 30       # Total: 30  number folded into text
show [1, 2] + [3]         # [1, 2, 3]  two lists into one
```

That third line matters: joining a number onto text needs no conversion. `"Total: " + 30` simply works.

Dividing by zero is an error, not `infinity`.

Numbers are shown tidied, so `0.1 + 0.2` displays as `0.3` rather than a long trail of digits.

---

7. Comparing

Comparisons are words, never symbols. Typing `>` or `==` gets you an error telling you the word to use instead.

WRITTEN	MEANS
`is`	exactly the same
`is not`	different
`is more than`	bigger
`is less than`	smaller
`is at least`	bigger or the same
`is at most`	smaller or the same

```
if score is 10
if name is not "Ada"
if age is at least 18
```

`is` compares by content, so two lists with the same items match:

```
show [1, 2] is [1, 2]      # true
```

Sizes can be compared for numbers, and for text alphabetically. **Capital letters and accents are ignored when ordering**, so text sorts the way a dictionary does:

```
show "apple" is less than "Banana"    # true
show sort_up(["banana", "Apple"])     # ["Apple", "banana"]
```

Note that ordering ignores capitals but `is` does not — `"Apple" is "apple"` is false. Matching is exact; only ordering is forgiving.

Comparing sizes of anything else is an error, with a message pointing you at `is`.

---

8. Logic and conditions

```
and    both must be true
or     either may be true
not    turns true into false
```

```
if age is at least 18 and name is not ""
  show "Allowed"
end

if not (score is 0)
  show "Started"
end
```

Everything must be true or false. There is no truthiness — an empty list, zero, and empty text are **not** false in Plain, they are simply not conditions:

```
if count(xs) is more than 0     # correct
if xs                           # error, with a message telling you the above
```

`IF`, `ELSE IF`, `ELSE`

```
if score is at least 9
  show "Publish"
else if score is at least 7
  show "Watch"
else
  show "Skip"
end
```

One `end` closes the whole chain. `else if` may repeat as many times as you like; `else` is optional and comes last.

---

9. Loops

`FOR EACH` — GO THROUGH A LIST

```
for each price in prices
  show price
end
```

Also works on text, one letter at a time:

```
for each letter in "abc"
  show letter
end
```

For a record, go through its field names:

```
for each field in keys(person)
  show field
end
```

`REPEAT` — A FIXED NUMBER OF TIMES

```
repeat 3 times
  show "tick"
end
```

`WHILE` — UNTIL A TEST STOPS BEING TRUE

```
set n to 1
while n is at most 5
  show n
  change n to n + 1
end
```

`STOP` — LEAVE A LOOP EARLY

```
for each n in [1, 2, 3, 4]
  if n is 3
    stop
  end
  show n
end
```

```
1
2
```

`stop` works in all three loops and leaves only the innermost one.

A loop that never finishes is stopped automatically after about four million steps or five seconds, with a message saying so. It will not hang.

---

10. Actions

`make` creates an action. `give` hands a value back.

```
make double(n)
  give n * 2
end

show double(21)        # 42
```

An action sees only what you pass in. It cannot read or alter anything outside itself, so its first line lists everything it uses:

```
set tax to 0.2
make total(amount)
  give amount * (1 + tax)     # error — tax is outside
end
```

> Line 3: `tax` exists outside this action, but actions can't see outside names. Pass it in instead.

```
make total(amount, tax)       # correct
  give amount * (1 + tax)
end
show total(100, 0.2)          # 120
```

Two useful consequences. Names are reusable — `n` can be an input to every action, since no two actions can see each other's. And nothing ever changes at a distance: to alter something outside, an action must give a value back and the caller must `change` it.

Actions can still call other actions, including themselves:

```
make factorial(n)
  if n is at most 1
    give 1
  end
  give n * factorial(n - 1)
end

show factorial(6)      # 720
```

Several inputs are separated by commas, and an action with no inputs still needs its brackets:

```
make greet()
  show "Hello"
end

greet()
```

`give` stops the action immediately, so it doubles as an early exit. An action that never reaches a `give` produces `nothing`. Calling with the wrong number of values is an error naming what it expected.

Actions are values, so they can be stored and passed on:

```
make double(n)
  give n * 2
end

make apply(fn, value)
  give fn(value)
end

show apply(double, 7)   # 14
```

11. Lists in detail

Items count from 1. The first item is `xs[1]`, matching how people already count.

```
set xs to ["a", "b", "c"]
show xs[1]             # a
show xs[3]             # c
show count(xs)         # 3
change xs[2] to "Z"    # xs is now ["a", "Z", "c"]
```

Asking for a position that doesn't exist tells you how many there are.

A list may hold anything, including other lists and records:

```
set people to [
  {name: "Ada", born: 1815},
  {name: "Alan", born: 1912}
]

for each person in people
  show person.name + " — " + person.born
end
```

A list written across several lines needs no continuation marks, as above.

Text also takes positions, giving one letter:

```
show "hello"[1]        # h
```

12. Records in detail

```
set person to {name: "Ada", born: 1815}
show person.name       # Ada
show keys(person)      # ["name", "born"]
show count(person)     # 2
```

Reading a field that doesn't exist lists the fields it does have, and suggests a correction if your spelling was close.

Records nest, and are read with dots all the way down:

```
set config to {owner: {name: "Ada", city: "London"}}
show config.owner.city      # London
```

Setting a field that doesn't exist yet adds it.

13. The 20 built-in actions

Always available. Nothing to import. **None of them ever changes what you give them** — they hand back a new value, so the original is untouched.

LISTS

ACTION	NEEDS	GIVES BACK
`count(thing)`	list, text or record	how many items, letters or fields
`add(list, item)`	a list and anything	a **new** list with the item on the end
`remove(list, position)`	a list and a position	a **new** list without that item
`first(list)`	a list with at least one item	the first item
`last(list)`	a list with at least one item	the last item
`reverse(list)`	a list or text	it, back to front
`has(collection, item)`	list, text or record	true or false
`sum(list)`	a list of numbers only	the total
`sort_up(list)`	all numbers, or all text	smallest or A–Z first
`sort_down(list)`	all numbers, or all text	largest or Z–A first
`sort_up(list, "field")`	a list of records	ordered by that field
`join(list, separator)`	a list and some text	one piece of text

```
set xs to [3, 1, 2]
show sort_up(xs)              # [1, 2, 3]
show sort_down(xs)            # [3, 2, 1]
show xs                       # [3, 1, 2] — untouched

change xs to add(xs, 4)       # to keep the result, change the name
show xs                       # [3, 1, 2, 4]
```

Sorting records needs the field name in quotes:

```
set people to [
  {name: "Grace", born: 1906},
  {name: "Ada", born: 1815}
]
show sort_up(people, "born")     # Ada first
show sort_down(people, "born")   # Grace first
```

Misspelling the field is an error naming the fields that exist, with a one-tap correction. It is checked when the line runs, not before.

TEXT

ACTION	NEEDS	GIVES BACK
`split(text, separator)`	two pieces of text	a list
`upper(text)`	text	text in capitals
`lower(text)`	text	text in lower case
`trim(text)`	text	text without leading or trailing spaces

NUMBERS

ACTION	NEEDS	GIVES BACK
`round(number)`	a number	the nearest whole number
`round(number, places)`	a number and how many places	rounded to that many decimals
`random(lowest, highest)`	two numbers	a whole number, either end possible

```
show round(17.12345, 2)    # 17.12
show round(2.5)            # 3
show round(-2.5)           # -3   — same distance either side of zero
```

Rounding gives a number, and numbers drop trailing zeros, so `round(17.1, 2)` shows as `17.1` rather than `17.10`. Fixed decimal places for display needs text formatting, which doesn't exist yet.

CONVERTING AND RECORDS

ACTION	NEEDS	GIVES BACK
`text(value)`	anything	it as text
`number(text)`	text that reads as a number	the number
`keys(record)`	a record	a list of its field names

14. Order of operations

From loosest to tightest:

1. `or`
2. `and`
3. comparisons — `is`, `is more than`, and the rest
4. `+` `-`
5. `*` `/` `%`
6. `not`, and negative signs
7. calls `()`, fields `.`, positions `[]`

So `2 + 3 * 4` is 14, and `a is 1 and b is 2` reads as expected. Use brackets whenever you'd rather not rely on remembering this:

```
show (2 + 3) * 4          # 20
```

---

15. Where names live

Two rules cover all of it.

Each block keeps its own new names. A name created inside an `if`, `for each`, `while` or `repeat` is forgotten at its `end`:

```
for each price in prices
  set found to price
end
show found
```

> Line 4: `found` only exists inside the `for` on line 1. Names made inside a block are forgotten at its `end`. Create it before the block instead: `set found to nothing`

Which is exactly how to write it — create it outside, alter it inside:

```
set found to nothing
for each price in prices
  change found to price
end
show found
```

That's why every accumulator starts before its loop. The loop's own name works the same way: `price` exists only for the loop.

Each action is sealed. It sees its inputs, anything it creates itself, the built-in actions, and other actions — nothing else. See section 10.

Together these give one rule worth remembering: **a name means one thing inside any single action, and one thing at the top level.** You'll almost never meet the error, because it only fires when you typed `set` where you meant `change`, or forgot a name was already in use.

16. Errors

Every error names the line, underlines the exact word, says what went wrong in a sentence, and where possible says what to do about it:

```
Line 10
show "Sum is " + totl
                 ^^^^
I don't know what "totl" is.
There is something called "total". Did you mean that?
```

The full set of things Plain will tell you:

- an unknown name, with the closest name you did define (your own names are offered before built-ins)
- a name that vanished at an `end`, naming the block it was created in
- `set` on a name that already exists, and `change` on one that doesn't
- a record field that doesn't exist, with the fields that do
- a list position past the end, with how many items there are
- arithmetic on the wrong kind of value, naming both sides
- an `if` or `while` test that isn't true or false, with a suggested comparison
- an action given the wrong number of values, naming what it expects
- a block never closed, pointing at the line where it opened

- text never closed with a second quote mark
- `=` where `set` or `change` was meant, and `>` or `==` where a word was meant
- an action reaching for a name outside itself
- a field name misspelled inside `sort_up` or `sort_down`
- dividing by zero
- text that can't be read as a number
- a loop that never finishes

Where the fix is unambiguous, the playground offers it as a button.

---

17. The complete word list

All 29. There are no others.

Instructions — `set`, `change`, `to`, `show`, `if`, `else`, `end`, `for`, `each`, `in`, `repeat`, `times`, `while`, `make`, `give`, `stop`

Comparing — `is`, `not`, `more`, `less`, `than`, `at`, `most`, `least`

Joining tests — `and`, `or`

Values — `true`, `false`, `nothing`

Six of these (`more`, `less`, `than`, `at`, `most`, `least`) only ever appear as part of a comparison such as `is more than`, so in practice there are 23 words to learn and six that come along with them.

Symbols — `+` `-` `*` `/` `%` `(` `)` `[` `]` `{` `}` `,` `.` `:` `"` `#`

18. What Plain does not have yet

Being honest about the edges, so nothing surprises you:

- **No file or network access.** A program cannot read a file or fetch a URL. This is the kernel, and it is the next thing to build.
- **No text formatting.** No padding, alignment or fixed decimal places, so a table of numbers comes out ragged.
- **No error handling.** No `try` / `catch`. An error stops the program.
- **No modules.** One program is one file; programs can't yet use each other, so there are no libraries.
- **No input.** No way to ask the person a question while running.
- **No classes.** Records hold values, not actions on those values. This may stay that way deliberately.
- **No async, threads or timing.** Nothing runs in the background.

SETTLED SINCE V0.1

All four rough edges from the first version are now closed:

1. **Names vanishing at `end`** — kept, because block scoping is correct and more expressive, but the error now names the block and says what to do instead.
2. **Mutation inconsistency** — settled. Nothing is ever changed in place. `add` and `remove` return new lists, matching `sort_up`, so `change xs to add(xs, 4)` is how you keep a result.
3. **Suggestions favouring built-ins** — fixed. Names you defined yourself are offered first.
4. **Asymmetric rounding** — fixed. `round(2.5)` is 3 and `round(-2.5)` is -3.

OPEN QUESTIONS

- **Checking earlier.** Quoted field names are checked when the line runs. A pass over the whole program before running would catch them sooner, along with unknown names and wrong argument counts. Planned for after the kernel.
- **Text formatting.** The shape of it isn't decided yet — likely a small group of actions rather than one.

19. A complete program

Everything in this document, used once:

```
# Work out which product line earned most last month.

set lines to [
  {name: "Monitors", units: 42, price: 189.99},
  {name: "Keyboards", units: 118, price: 34.50},
  {name: "Cables", units: 306, price: 4.25}
]

make revenue(line)
  give line.units * line.price
end

set totals to []

for each line in lines
  set earned to revenue(line)
  change totals to add(totals, earned)
  show line.name + ": " + round(earned, 2)
end

show ""
show "Total:", round(sum(totals), 2)
show "Average per line:", round(sum(totals) / count(lines), 2)

show ""
show "Best first:"
for each line in sort_down(lines, "units")
  show "  " + line.name + " — " + line.units + " units"
end
```

```
Monitors: 7979.58
Keyboards: 4071
Cables: 1300.5

Total: 13351.08
Average per line: 4450.36

Best first:
  Cables — 306 units
  Keyboards — 118 units
  Monitors — 42 units
```

Note the shape: `set totals to []` before the loop, `change` inside it, and `revenue` taking everything it needs as an input.